

Tutorial 3

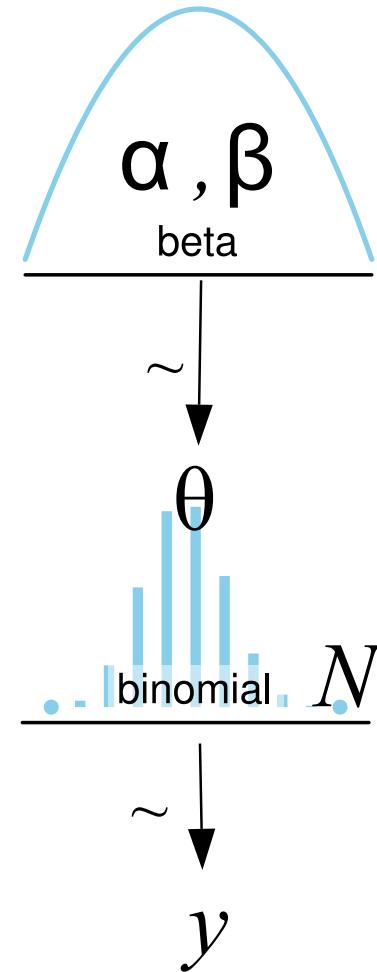
Statistical Computation and Analysis
Spring 2026

Tutorial Outline

- Graphical models
- Probabilistic programming languages – PyMC
- Sampling
- Posterior distribution
 - (Bayesian) Estimation
 - HDI
- Frequentist estimation
 - MLE
 - CI

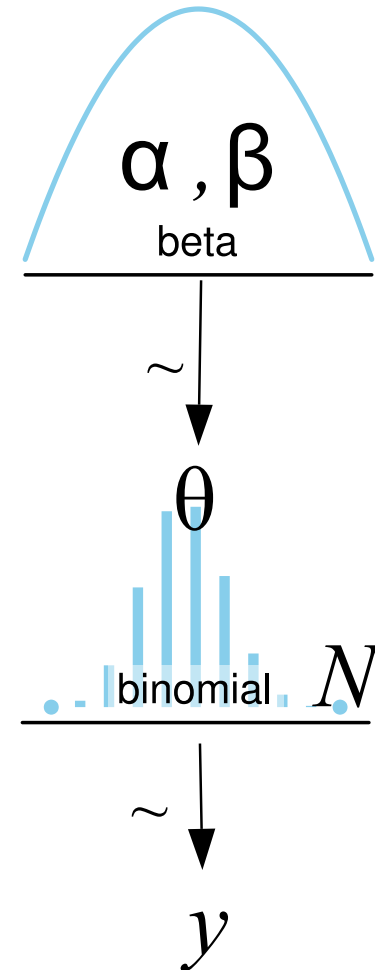
Graphical model

- Representing our models with diagrams



Graphical model

- Data at the bottom
 - y coin flips
- Likelihood in the middle
 - N coin flips with an average probability of θ of being 'head'
 - This is our model of how the data was produced
- Prior at the top
 - Reflects our uncertainty about θ when the experiment began.

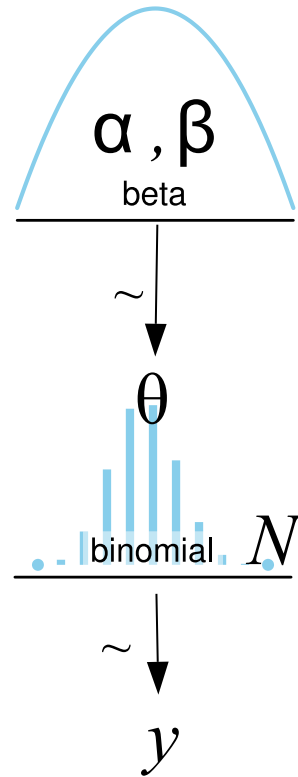


Three Representations of the Same Idea

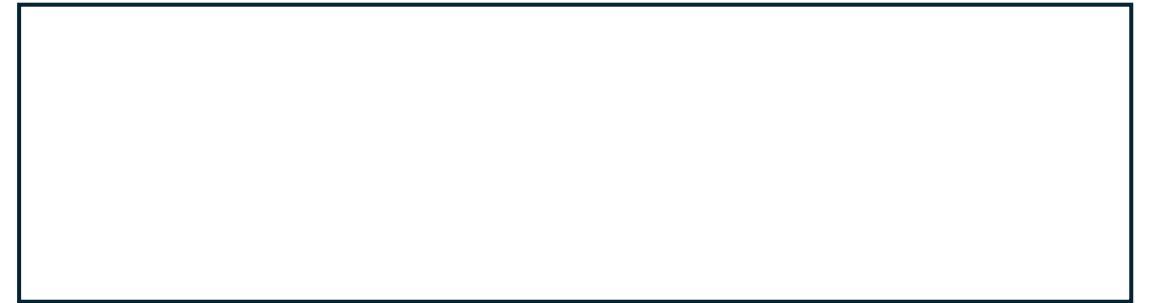
Equations

$$y \sim \text{Binom}(\theta, N)$$
$$\theta \sim \text{Beta}(\alpha, \beta)$$

Graphical model



PPL



Probabilistic Programming Languages (PPL)

- Fully probabilistic models often lead to analytically intractable expressions
- Probabilistic Programming Languages
 - Allow clear separation between model creation and inference
 - Users specify a full probabilistic model by writing a few lines of code
 - The PPL automatically builds the model and uses algorithms to draw samples from the posterior.
 - These samples are then used to estimate posterior summaries, uncertainty intervals, and diagnostic

PyMC

A PPL for Python

- **PyMC** defines the model.
- **ArviZ** helps to analyzes posterior samples.
- **Preliz** helps choose priors.



The screenshot shows the PyMC website's home page. At the top, there is a navigation bar with links: [Home](#) (underlined), [Examples](#), [Learn](#), [API](#), [Community](#), and [Contributing](#). On the right side of the navigation bar is a search bar with a magnifying glass icon and the text "Search". Below the navigation bar, there is a breadcrumb trail: [Home](#). The main content area features a large illustration of a rocket launching from a base of nodes, with the text "PYMC" in large, bold, blue letters to its right. Below this illustration, a paragraph states: "PyMC is a probabilistic programming library for Python that allows users to build Bayesian models with a simple Python API and fit them using Markov chain Monte Carlo (MCMC) methods." Further down, there is a section titled "Features" with a paragraph stating: "PyMC strives to make Bayesian modeling as simple and painless as possible, allowing users to focus on their problem rather than the methods."

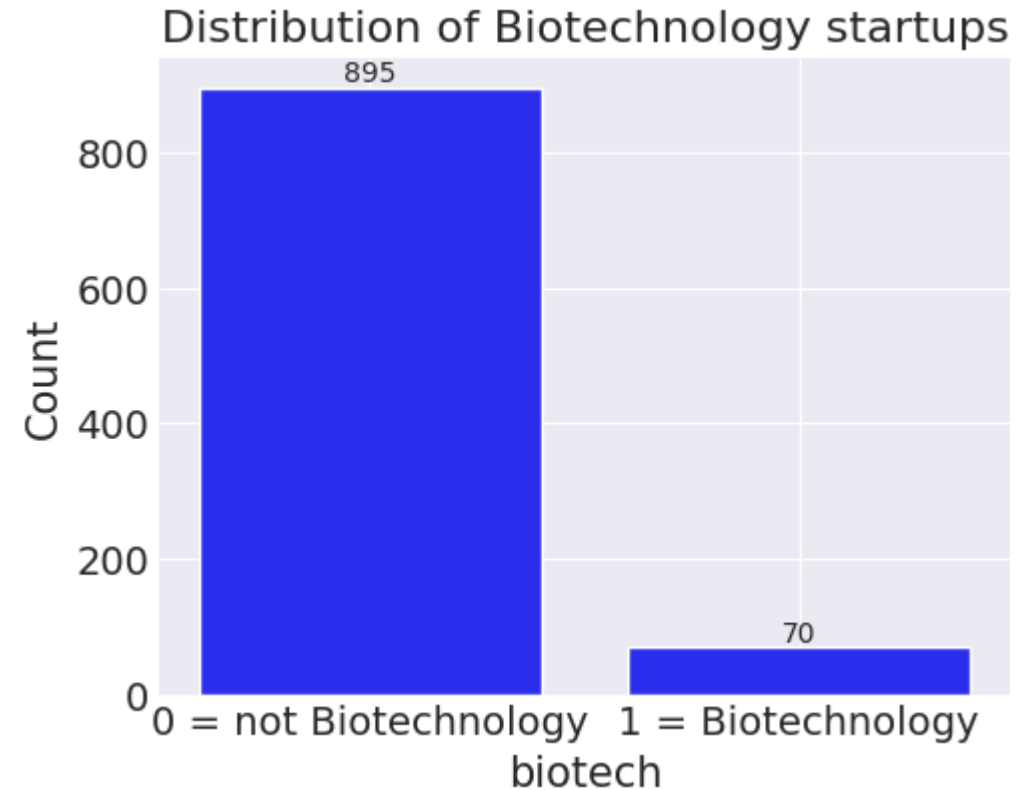
Translating a Graphical Model to a PPL

- Biotechnology startups

	name	status	country_code	funding_total_usd	category_list
0	eMotion Technologies	operating	ISR	10000	Games
1	Microstrip Planar Antennas	operating	ISR	800000	Messaging
2	Integrity Applications	ipo	ISR	22607446	Health Care
3	RBNanDoors	operating	ISR	11000000	Design Manufacturing Public Safety Security
4	Mobilicom	operating	ISR	Nan	Commerce Telecommunications Wireless Mobile

Translating a Graphical Model to a PPL

- Biotechnology startups
- Variable
 - biotech = 1 if the startup **category** is "Biotechnology"
 - 0 otherwise



Translating a Graphical Model to a PPL

- Biotechnology startups
- 2 possible models

$y \sim \text{Binom}(\theta, N)$  y - Number of startup in the biotechnology category

$y_i \sim \text{Bernulli}(\theta)$  y_i - Startup category is to biotechnology (one company, binary)

$\theta \sim \text{Beta}(\alpha, \beta)$  Probability of biotechnology startup

Translating a Graphical Model to a PPL

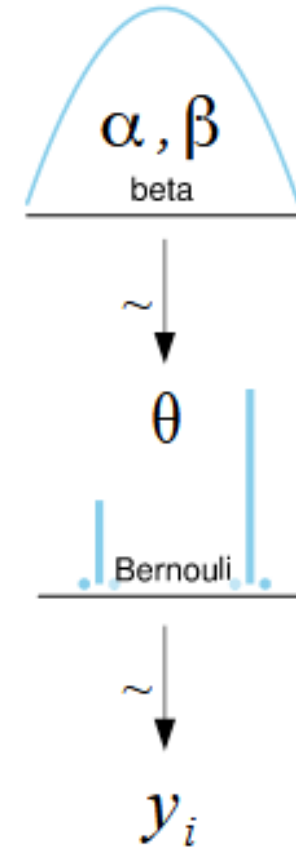
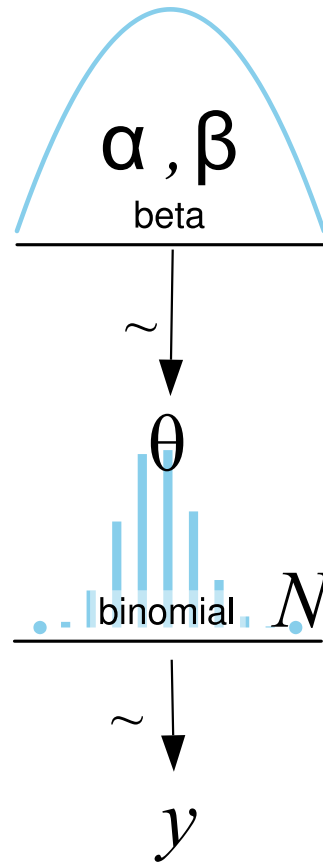
- Biotechnology startups

- 2 possible models

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y \sim \text{Binom}(\theta, N)$$

$$y_i \sim \text{Bernoulli}(\theta)$$



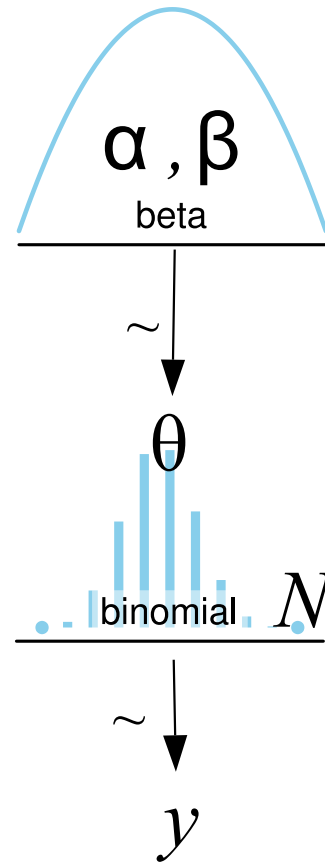
Translating a Graphical Model to a PPL

- Biotechnology startups
- 2 possible models

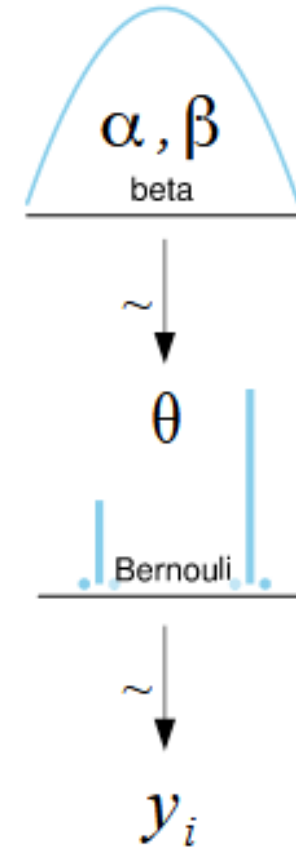
$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y \sim \text{Binom}(\theta, N)$$

$$y_i \sim \text{Bernoulli}(\theta)$$



Use when data is a sum of
Bernoulli outcomes



Use when data is a vector of
Bernoulli outcomes

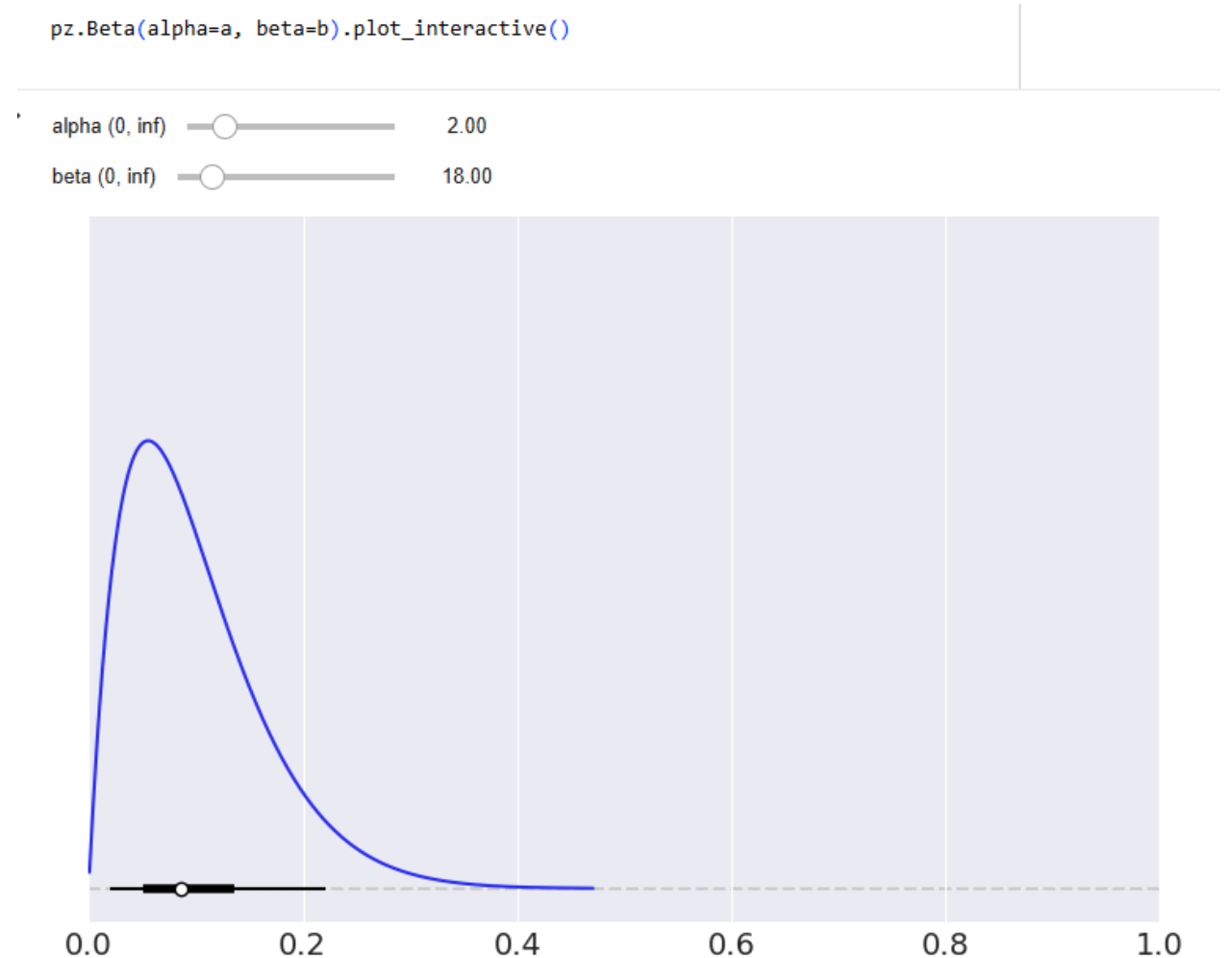
Translating a Graphical Model to a PPL

- Biotechnology startups

- Choosing a prior

$$\theta \sim \text{Beta}(2, 18)$$

$$y_i \sim \text{Bernulli}(\theta)$$



Translating a Graphical Model to a PPL

■ Biotechnology startups - PPL

```
# Choosing a prior
a = 2
b = 18

# First PyMC model
with pm.Model() as biotech_model:
    theta = pm.Beta("theta", alpha=a, beta=b)
    y = pm.Bernoulli("y", p=theta, observed=data)

idata = pm.sample(1000, chains=4)
```

Line 1: creates a container for our model. Everything inside the with block will be automatically added to `biotech_model`.

Line 2: prior

Line 3: likelihood

Line 4: computes the posterior using numerical methods.

Translating a Graphical Model to a PPL

■ Biotechnology startups – Additions to the simple model

```
# coords - naming our data
coords = {"startup": np.arange(len(data))}

# Choosing a prior
a = 2
b = 18

# First PyMC model
with pm.Model(coords=coords) as biotech_model:
    theta = pm.Beta("theta", alpha=a, beta=b)
    y = pm.Bernoulli("y", p=theta, observed=data, dims='startup')

idata = pm.sample(1000, chains=4, random_seed=17)
```

Coords & dims – for now, help us name the data in the idata object

Sampling

- The use of numerical methods solves the challenge of posteriors we cannot compute.
- However, what we get are samples from the posterior, rather than the posterior itself.
- Every time we run the inference, the samples will change.
- **If the inference process works as expected, the samples will be representative of the posterior distribution, and we will obtain the same conclusion from any of those samples.**

Sampling

■ Biotechnology startups

```
# coords - naming our data
coords = {"startup": np.arange(len(data))}

# Choosing a prior
a = 2
b = 18

# First PyMC model
with pm.Model(coords=coords) as biotech_model:
    theta = pm.Beta("theta", alpha=a, beta=b)
    y = pm.Bernoulli("y", p=theta, observed=data, dims='startup')

idata = pm.sample(1000, chains=4, random_seed=17)
```

Random seed – Help us get the same “random” results everytime we run the model

Sampling

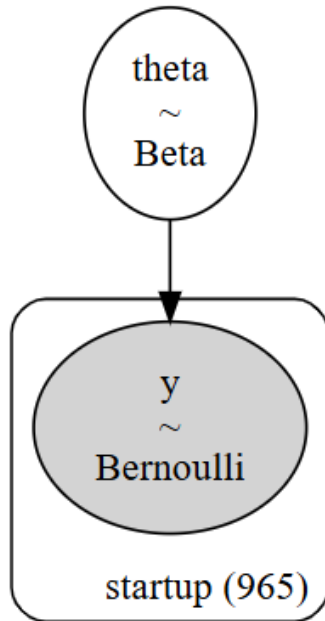
```
with pm.Model(coords=coords) as biotech model:  
    theta = pm.Beta("theta", alpha=a, beta=b)  
    y = pm.Bernoulli("y", p=theta, observed=data,  
                    dims='startup')  
  
idata = pm.sample(1000, chains=4, random_seed=17)
```

- **Posterior sampling** - generating draws that approximate the posterior distribution
- **Draw** - one set of parameter values generated from the posterior distribution.
- **Chain** - a sequence of posterior draws generated by one run of the sampler
- **InferenceData object** - the object PyMC returns to store posterior draws and related outputs

Graphical visualizer

- A tool for displaying the dependency structure of a PyMC model

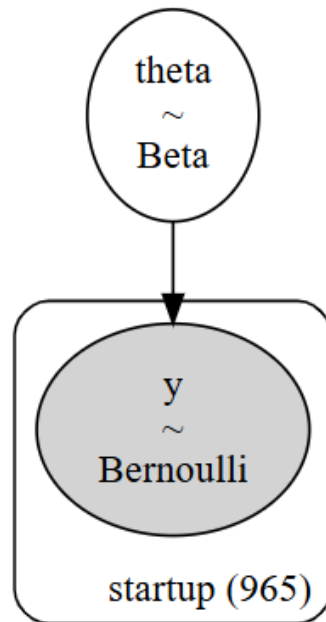
```
pm.model_to_graphviz(biotech_model)
```



Graphical visualizer

- A tool for displaying the dependency structure of a PyMC model

```
pm.model_to_graphviz(biotech_model)
```



In words (bottom up):

The dataset contains 965 startups.

Each startup outcome y_i (Biotechnology or not) is modeled as $Bernoulli(\theta)$.

The probability θ is drawn from a Beta prior.

InferenceData Object

- **idata** – what we get back from the inference process
- Container for all the data generated by PyMC.

idata

arviz.InferenceData

▶ posterior

▶ sample_stats

▶ observed_data

- Xarray object
- We will use the Arviz library to analyze the posterior

InferenceData Object

■ Observed data

arviz.InferenceData

► posterior

► sample_stats

▼ observed_data

xarray.Dataset

► Dimensions: (startup: 965)

Number of samples collected (965)

▼ Coordinates:

startup (startup) int64 0 1 2 3 4 5 ... 960 961 962 963 964

Their coordinates – here just 0-964

▼ Data variables:

y (startup) int64 0 0 0 0 0 0 0 ... 0 0 0 0 0 0 0

The values:
1 for biotechnology startups
0 otherwise

▼ Attributes:

created_at : 2026-04-29T13:46:34.213092+00:00
arviz_version : 0.22.0
inference_library : pymc
inference_librar... 5.28.4

InferenceData Object

■ Posterior

arviz.InferenceData

▼ posterior

xarray.Dataset

► Dimensions: (chain: 4, draw: 1000)

▼ Coordinates:

chain	(chain)	int64	0 1 2 3
draw	(draw)	int64	0 1 2 3 4 5 ... 995 996 997 998 999

4 chains of 1000 samples (draws) each

▼ Data variables:

theta	(chain, draw)	float64	0.08097 0.07666 ... 0.0822 0.08505
-------	---------------	---------	------------------------------------



The theta values

▼ Attributes:

created_at : 2026-04-29T13:46:34.158090+00:00
arviz_version : 0.22.0
inference_library : pymc
inference_librar... 5.28.4
sampling_time : 5.0950927734375
tuning_steps : 1000

► sample_stats

► observed_data

Posterior

- We can also get the data as samples instead of as separate chains using `az.extract`

```
posterior_samples = az.extract(idata)
posterior_samples
```

xarray.Dataset

► Dimensions: (sample: 4000)

▼ Coordinates:

sample	(sample)	object	MultiIndex	 
chain	(sample)	int64	0 0 0 0 0 0 0 ... 3 3 3 3 3 3 3 3	 
draw	(sample)	int64	0 1 2 3 4 5 ... 995 996 997 998 999	 

▼ Data variables:

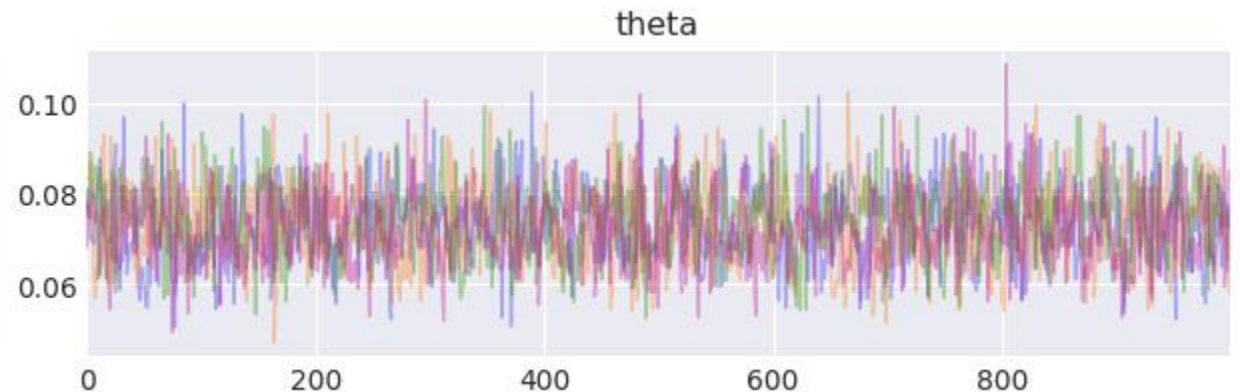
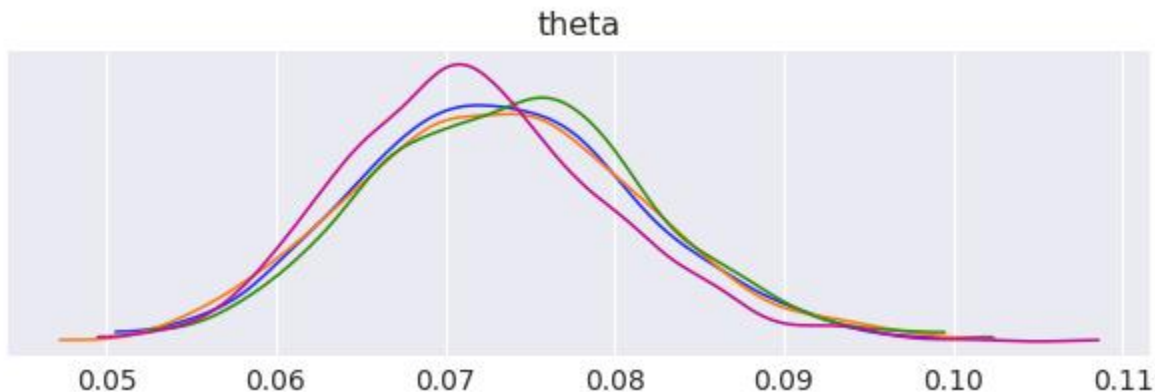
theta	(sample)	float64	0.08097 0.07666 ... 0.0822 0.08505	 
--------------	----------	---------	------------------------------------	---

▼ Attributes:

created_at : 2026-04-29T13:46:34.158090+00:00
arviz_version : 0.22.0
inference_library : pymc
inference_librar... 5.28.4
sampling_time : 5.0950927734375
tuning_steps : 1000

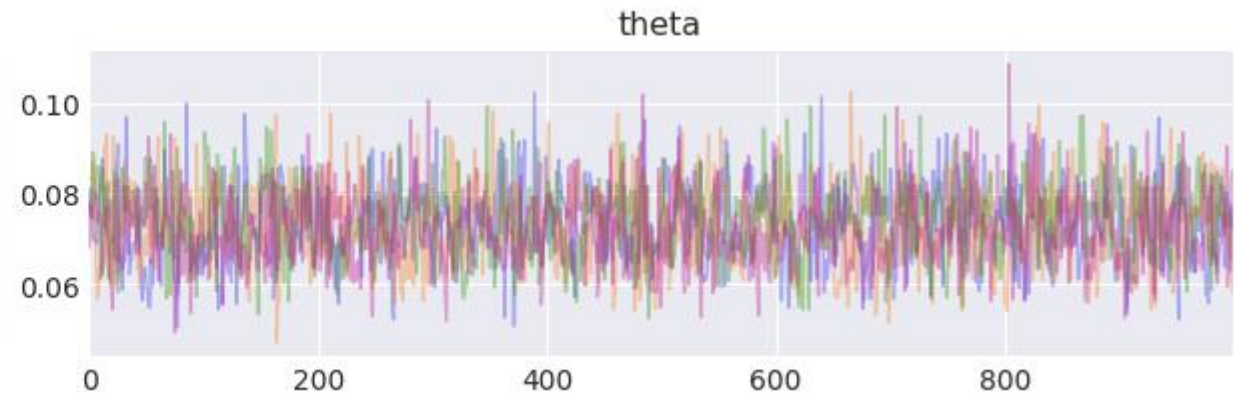
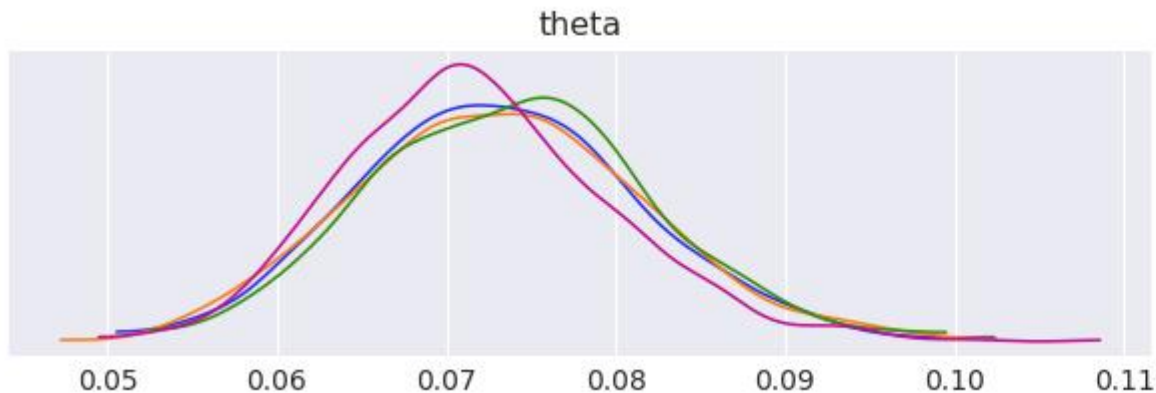
Analyzing the Posterior

- Look at the results
- Trace plot - a plot of sampled draws across chains and iterations
 - `az.plot_trace(idata)`
 - `az.plot_trace(idata, compact=False)` if you want the colorful chains



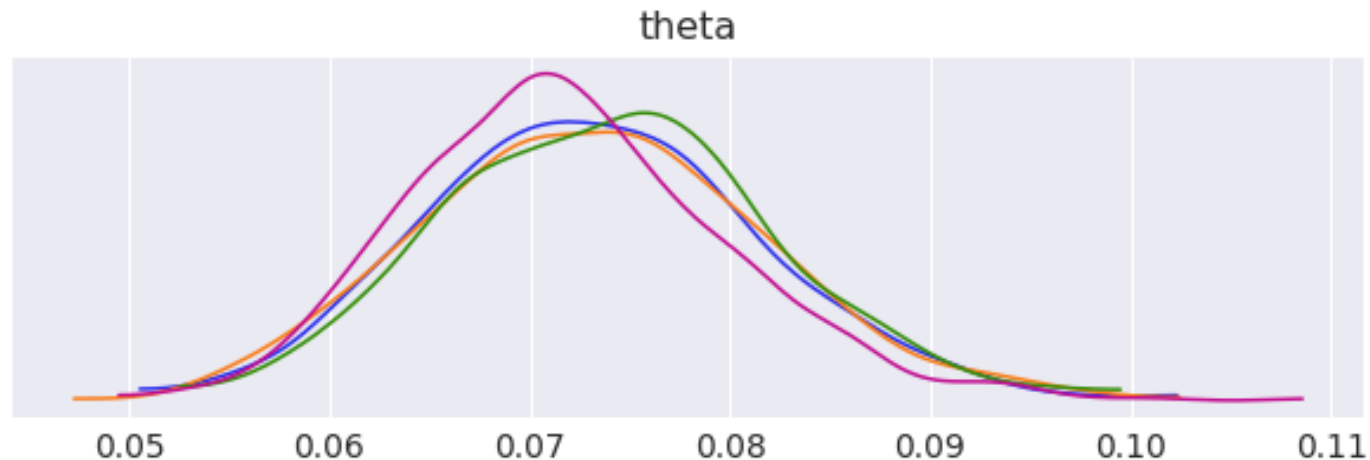
Analyzing the Posterior

- **Left:** the four chains showing the values of the posterior distribution (KDE plot = Kernel Density Estimation)
- **Right:** For now – the overlap between the four chains (colors) shows that the sampler is working.



Analyzing the Posterior

- We want all the chains to overlap as much as possible.
 - How much?
 - We'll learn statistics with thresholds we can calculate
 - For now, It looks pretty good.

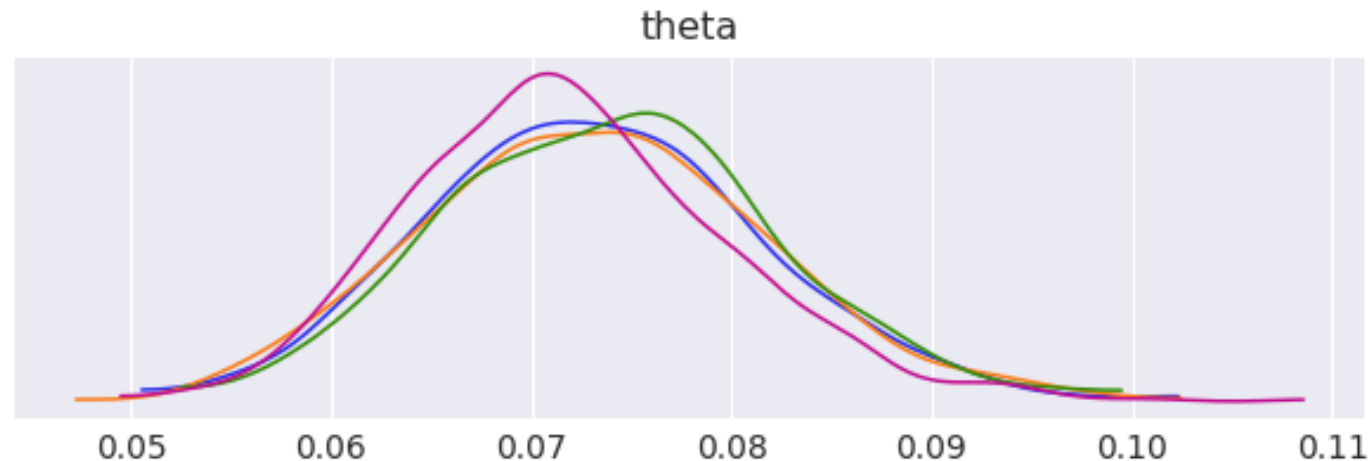


Analyzing the Posterior

- How close are the means of the different chains?

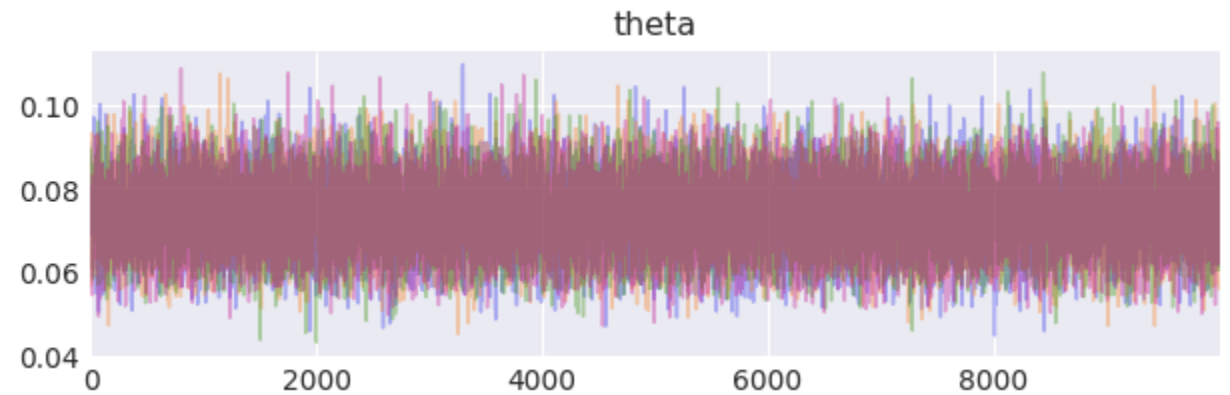
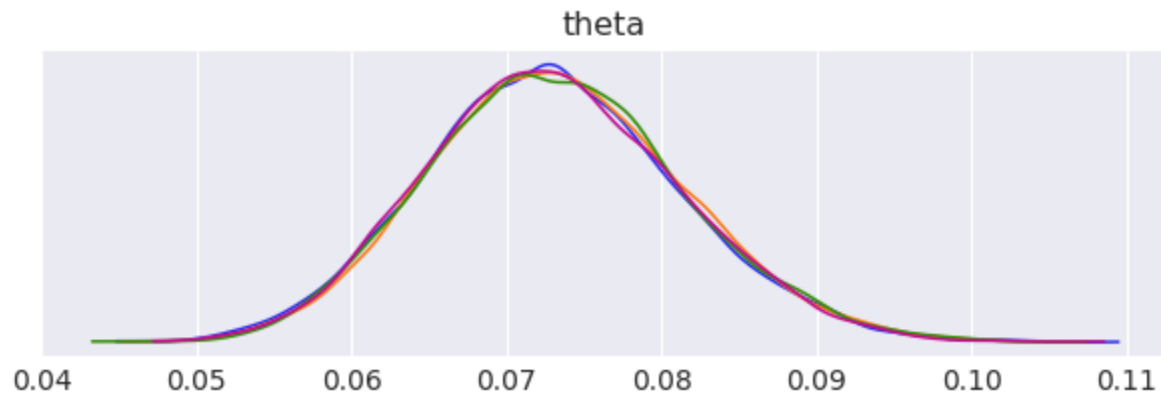
```
ms = idata.posterior.mean(dim = 'draw')  
print(f'The means of the 4 chains are: {np.round(ms.theta.to_numpy(), 3)}')
```

The means of the 4 chains are: [0.073 0.073 0.074 0.072]



Analyzing the Posterior

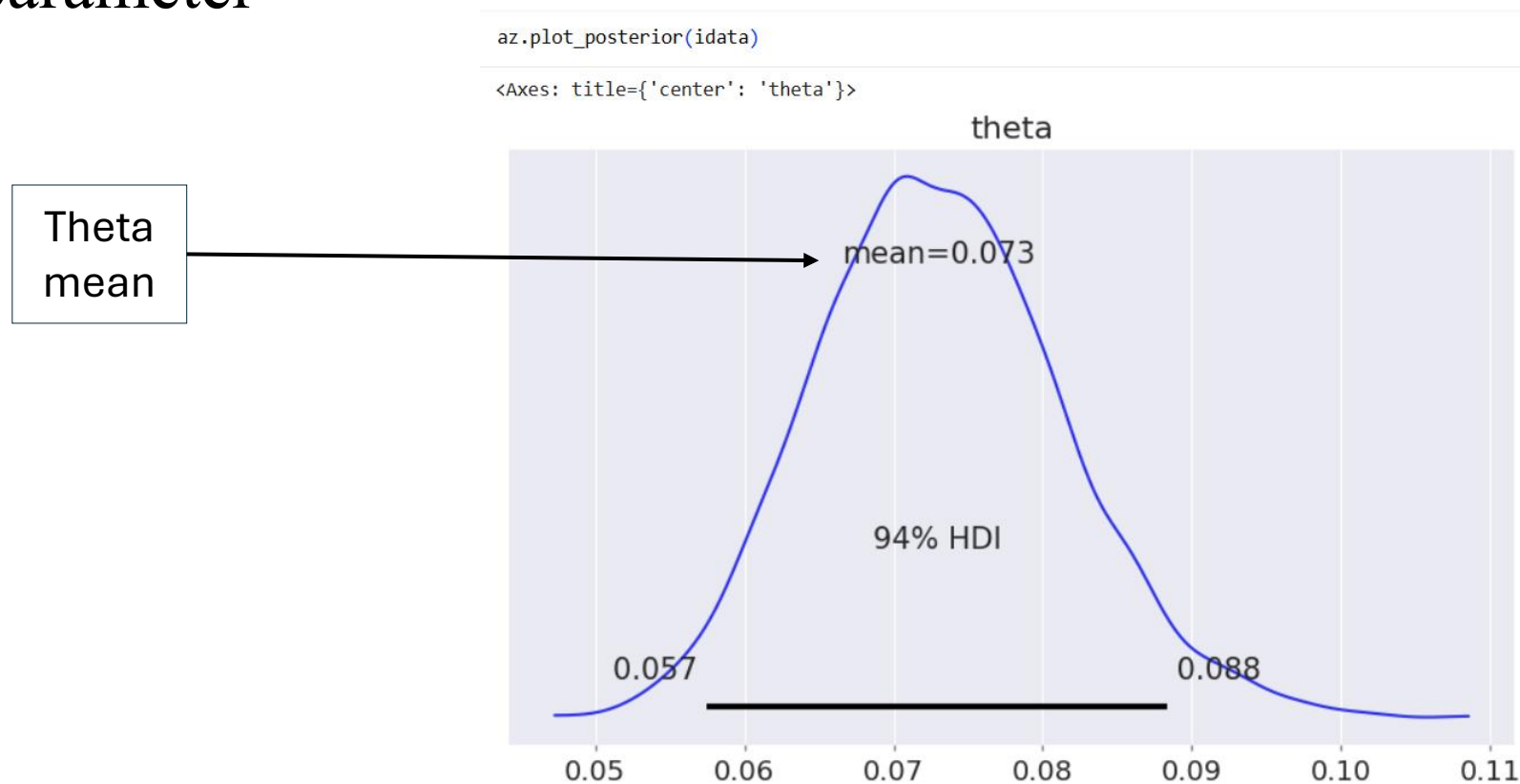
- How can we get the chains to look more similar?
 - Sample more (1,000 $\rightarrow$ 10,000)



The means of the 4 chains are: `[0.0729 0.0733 0.0733 0.073 ]`

Analyzing the Posterior

- Posterior plot -a plot summarizing the posterior distribution of a parameter

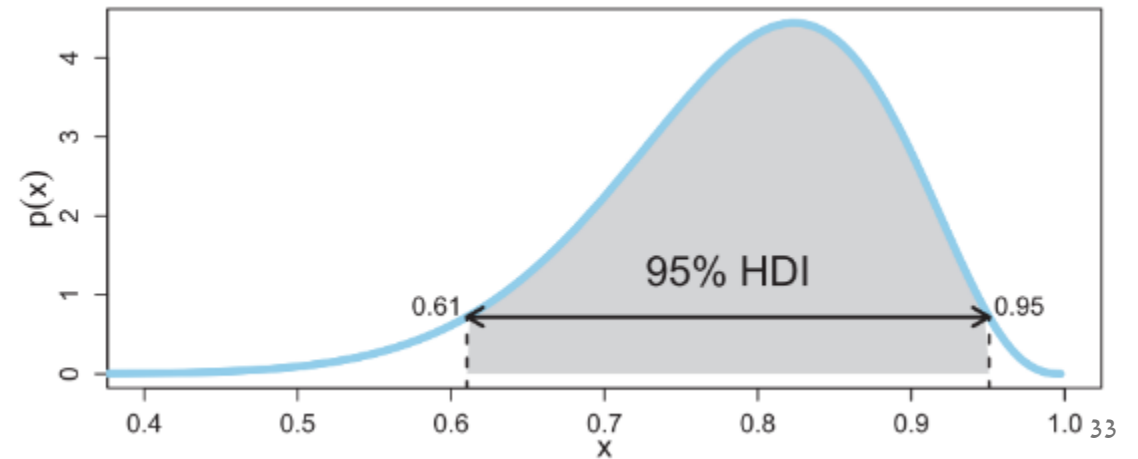
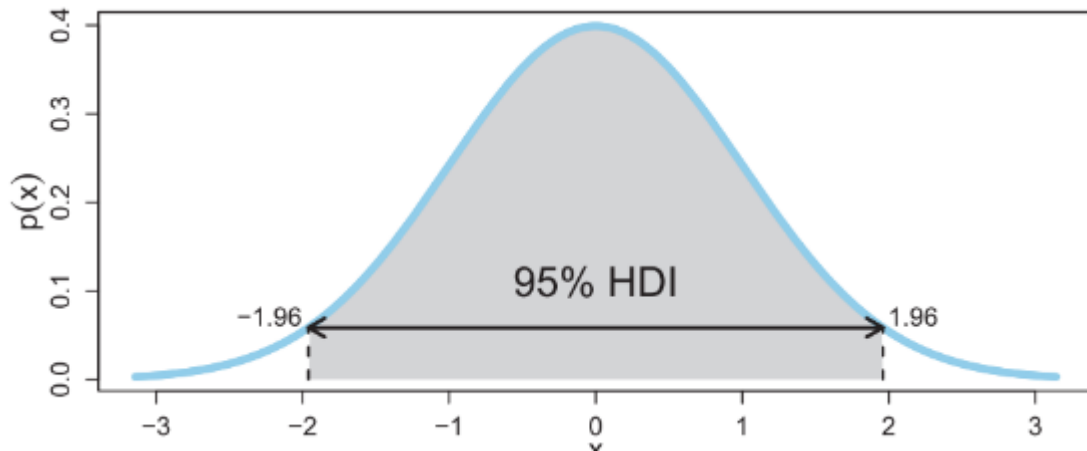


Highest Density Interval (HDI)

- We computed the mean
- We can also compute other statistics, such as the HDI
- HDI - the highest density interval, an interval containing the most credible posterior values.

Highest Density Interval (HDI)

- There are a lot of intervals like this.
- The HDI is the shortest interval containing $Z\%$ of the distribution.
- Equivalent: the probability of all values inside the HDI are higher than those out of it.



Highest Density Interval (HDI)

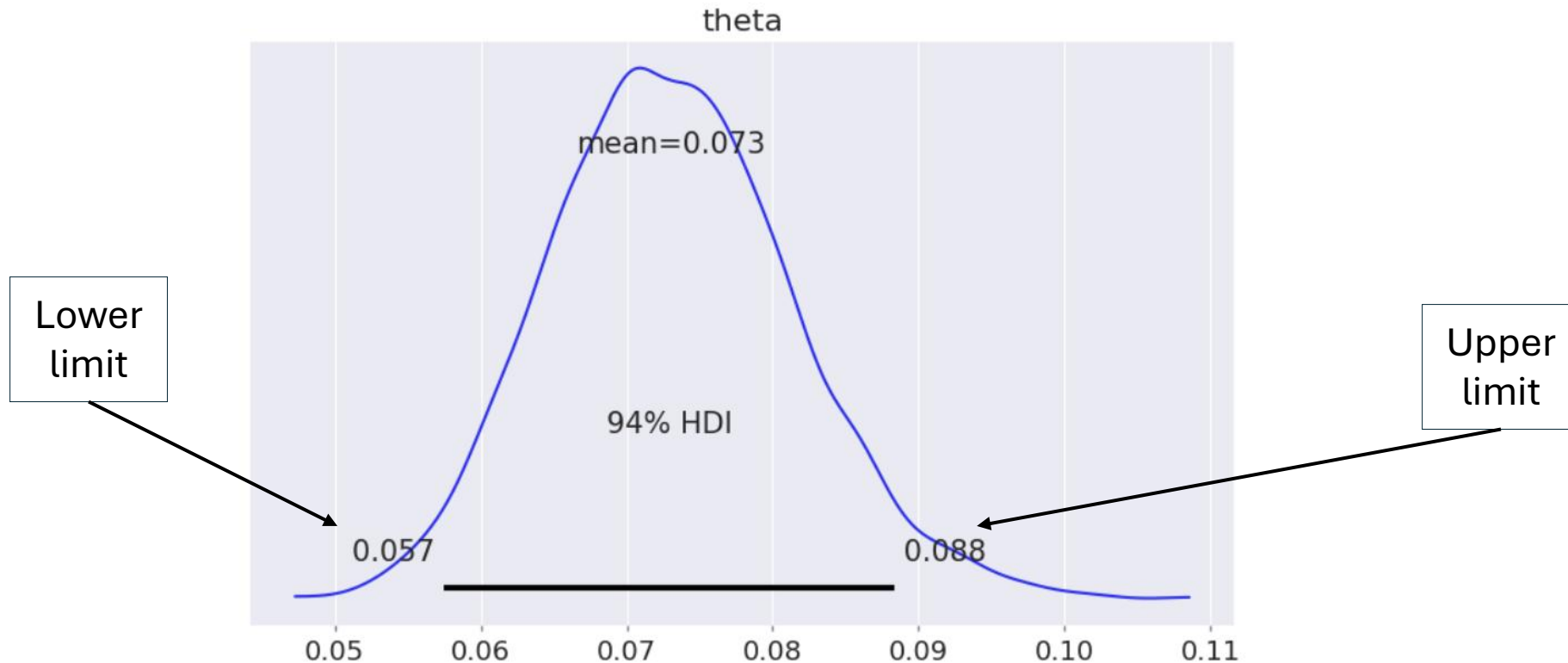
- The default in Arviz is the 94% HDI, so that's what we'll generally use.

Highest Density Interval (HDI)

- The default in Arviz is the 94% HDI, so that's what we'll generally use.

```
az.plot_posterior(idata)
```

```
<Axes: title={'center': 'theta'}>
```



Highest Density Interval (HDI)

- The default in Arviz is the 94% HDI, so that's what we'll generally use.

```
az.summary(idata, kind="stats")
```

	mean	sd	hdi_3%	hdi_97%
theta	0.073	0.008	0.057	0.088



Lower
limit

Upper
limit

Your turn

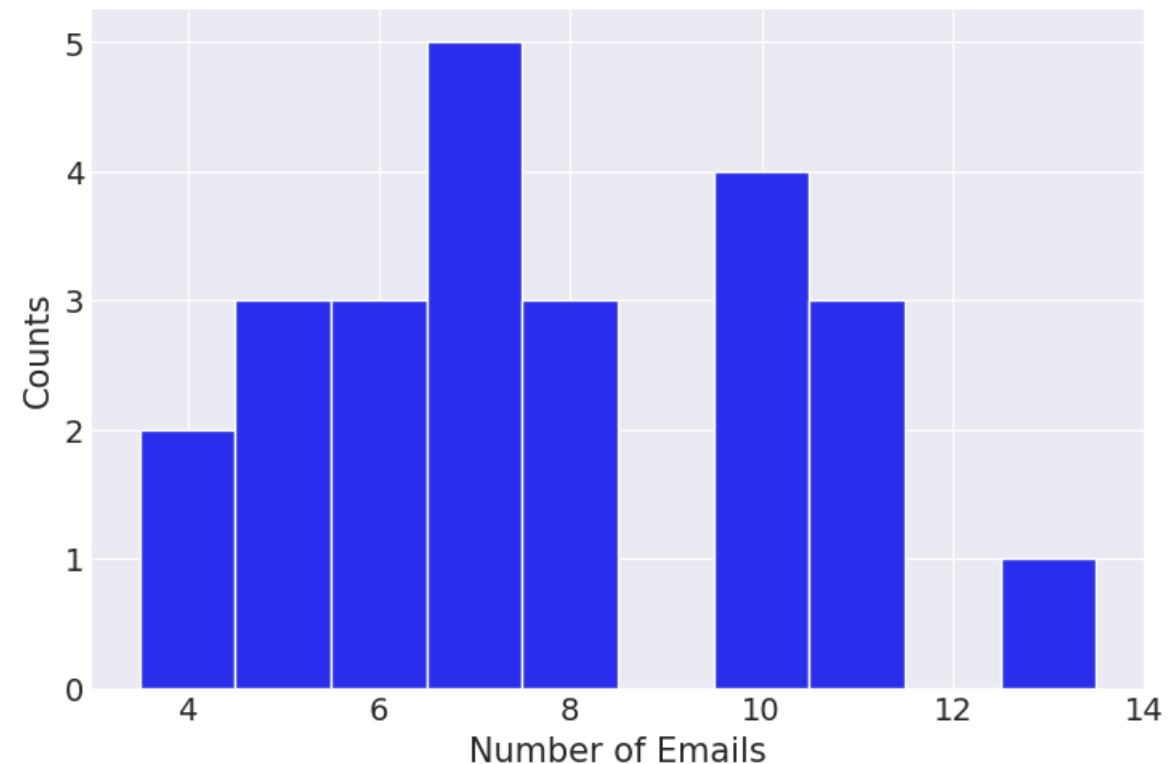
- Model the number of emails a student gets in a day:
 - Poisson likelihood
 - Gamma prior on λ (remember why?)

```
#simulate data
email_data = rng.poisson(lam=8, size=24)
print(email_data)
```

```
n = len(email_data)
print(f'There are {n} participants')
```

```
S = np.sum(email_data)
print(f'There are a total of {S} emails')
```

```
[10  7  4  7 14  4 11 11  6  7  6  8 11  8 10  7  5  7 10  5 10  8  6  5]
There are 24 participants
There are a total of 187 emails
```



Exercise:

- Go to the tutorial code notebook and:
 1. Choose a prior
 2. Use PyMC to create and sample the model
 3. Visualize the model in a graph
 4. Analyze the posterior using Arviz

Frequentist vs Bayesian philosophies

- The Bayesian view - data as updating our uncertainty about parameters
- Frequentist view - uncertainty as arising from repeated random datasets generated by a procedure.

In other words:

- Bayesian uncertainty - uncertainty about parameters or models given the data and assumptions
- Frequentist uncertainty - uncertainty defined through repeated sampling behavior of a procedure

Frequentist estimation

Important Terms (reminder)

- Statistic - any quantity calculated from data
- Estimator - a statistic used to guess a real parameter value
- Sampling distribution - the distribution of an estimator across repeated datasets



Powerful open-source Python library for scientific and technical computing.

Builds on NumPy and provides tools for:

- Optimization (e.g., `scipy.optimize`)
- Statistics (e.g., `scipy.stats`)
- Linear algebra (e.g., `scipy.linalg`)
- Signal processing, Fourier transforms
- Numerical integration and interpolation

Example Use Cases:

- Fitting models with `optimize.minimize` or `optimize.minimize_scalar`
- Calculating probabilities and distributions with `stats.models`
- Solving equations or differential equations

Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation (MLE)

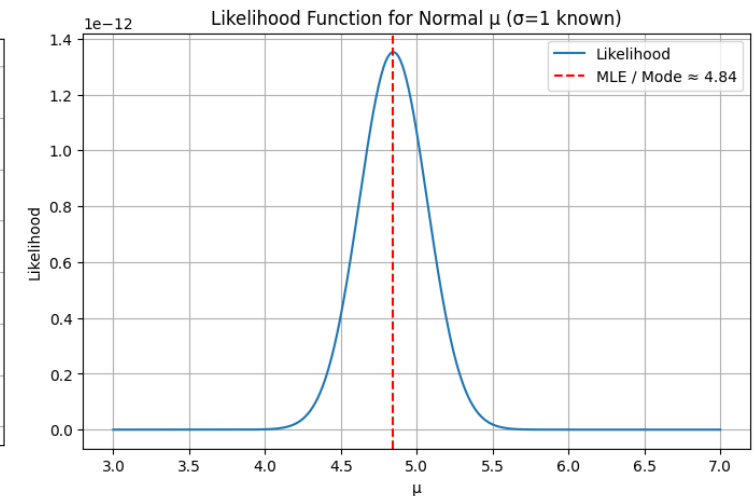
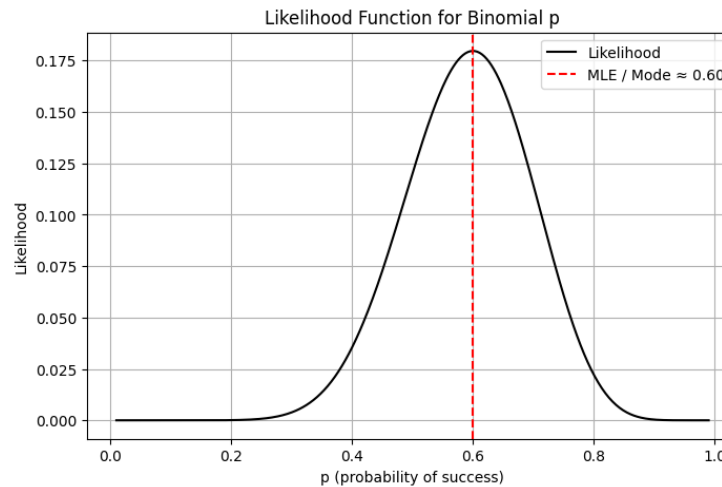
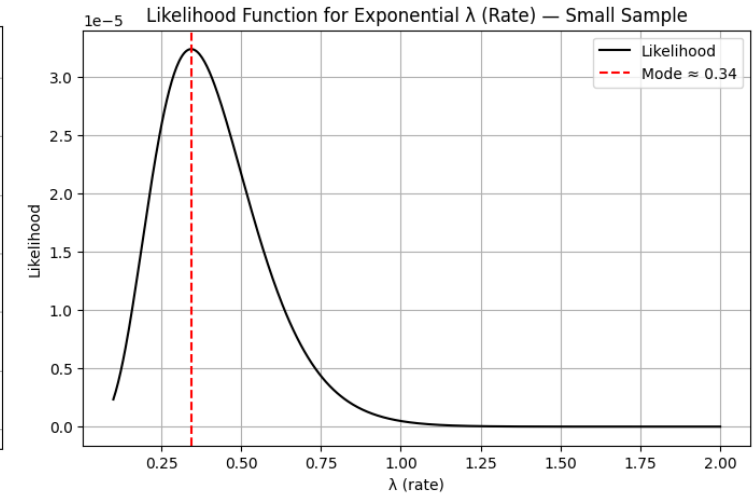
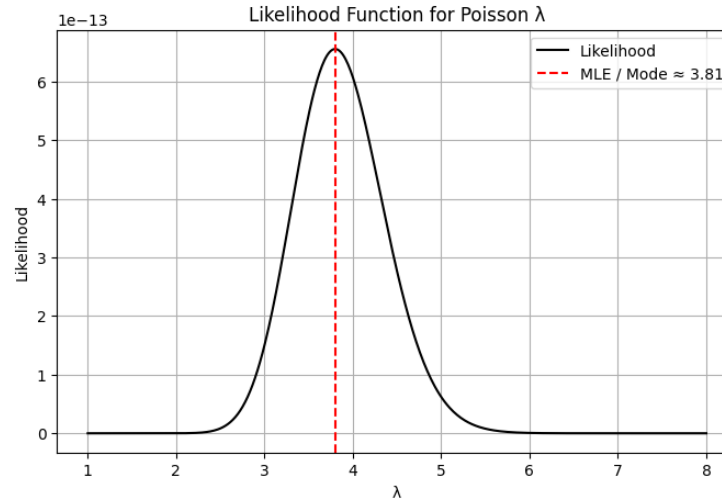
- A technique used for estimating the parameters of a given distribution, using some observed data
- Estimate of parameter that maximizes the likelihood function:

$$\hat{\theta}_{MLE} = \arg \max L(\theta)$$

$$L(\theta) = P(data | \theta) \text{ when } data = x_1, x_2, \dots, x_n$$

Maximum Likelihood Estimation (MLE)

- MLE is the mode of the likelihood



Analytic MLE

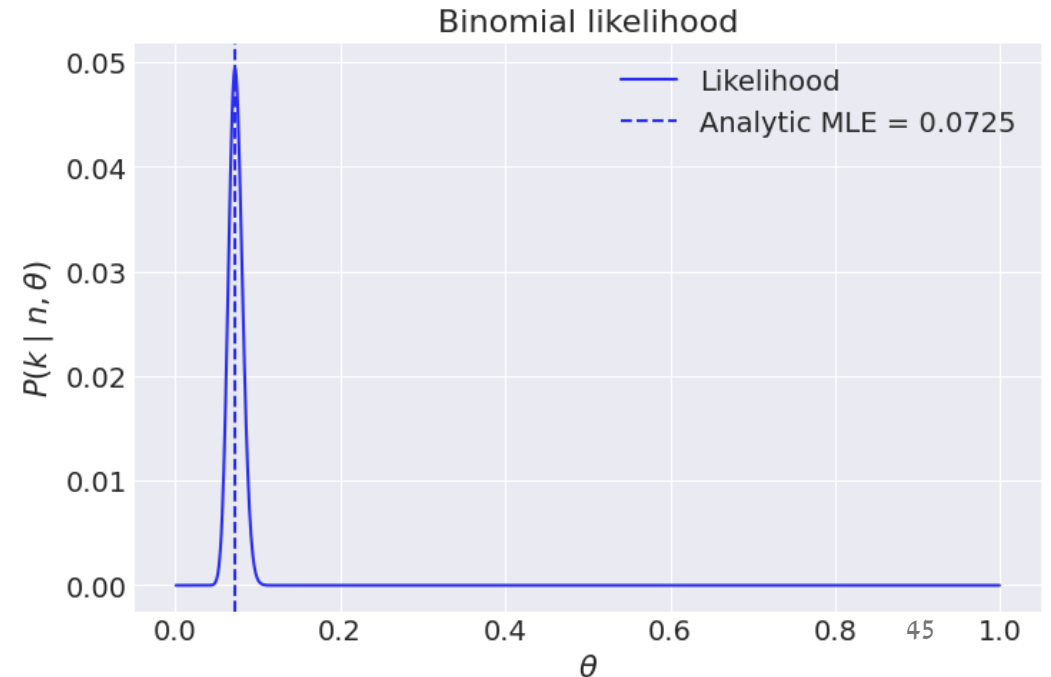
- **Analytic solution** means solving the MLE by hand - finding a closed-form expression that maximizes the likelihood.
- This is possible for simple models like the Binomial distribution:

```
Number of startups n 965
Number of Biotechnology startups k 70

theta_hat_analytic = k / n

print("Analytical MLE for the Binomial model:")
print(f"theta_hat = k/n = {k}/{n} = {theta_hat_analytic}")
```

```
Analytical MLE for the Binomial model:
theta_hat = k/n = 70/965 = 0.07253886010362694
```

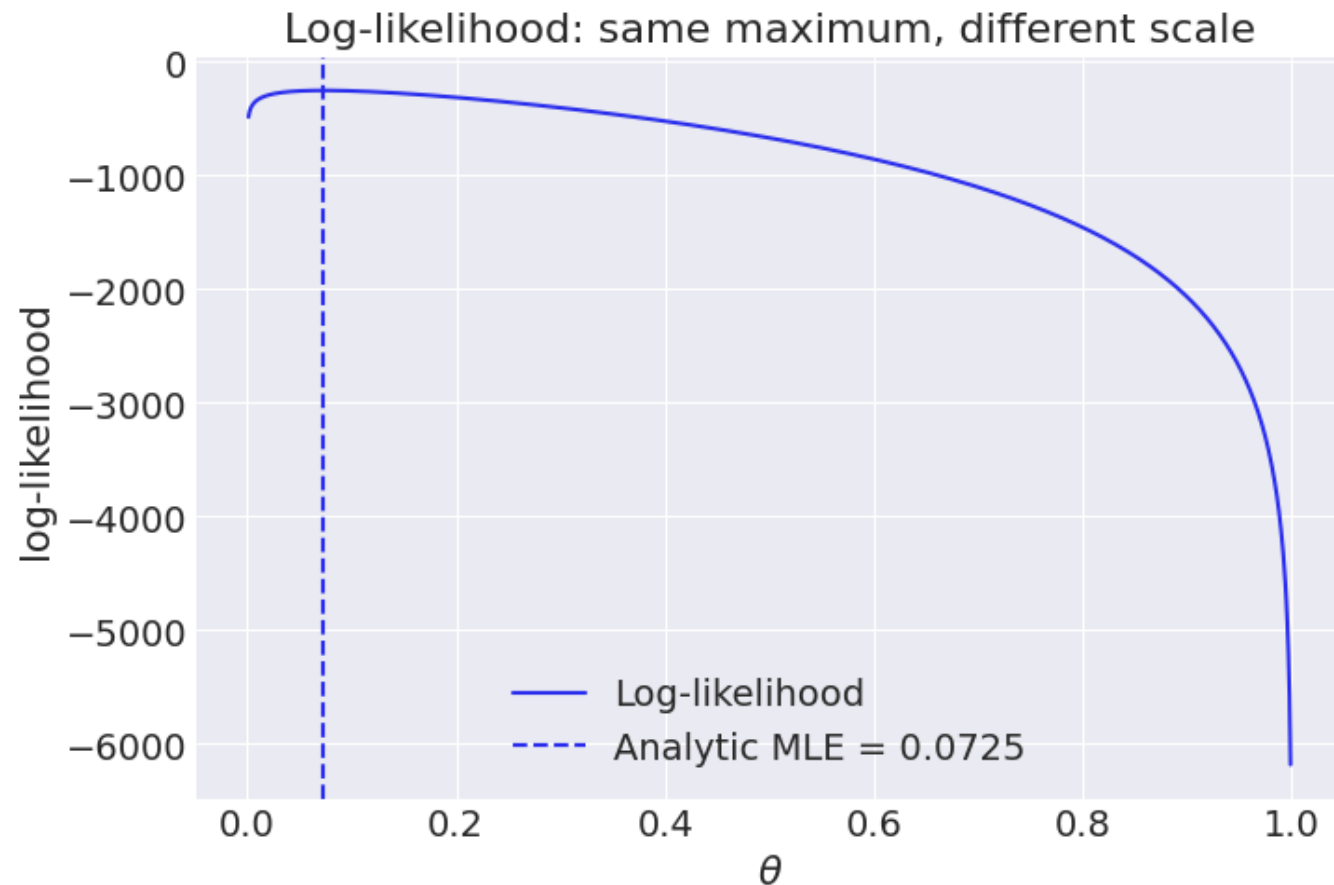


Log-likelihood

- We use the log-likelihood instead of the likelihood because it is numerically stable, mathematically convenient, and gives the same MLE.
- This is standard practice in both frequentist and Bayesian inference.

Log-likelihood

- Same MLE, different scale



Computational MLE

- **Computational solution** is used when the MLE can't be written down directly.
- We define a likelihood or log-likelihood and use numerical optimization to find the parameter value that maximizes it.

Computational MLE

```
from scipy.optimize import minimize_scalar
```

```
def negative_binomial_loglikelihood(theta, k, n):  
    """  
    Negative log-likelihood for numerical optimization.  
    """  
    return -binomial_loglikelihood(theta, k, n)
```

Creating a negative log-likelihood function

```
opt_result = optimize.minimize_scalar(  
    negative_binomial_loglikelihood,  
    bounds=(0.001, 0.999),  
    args=(k, n),  
    method="bounded"  
)
```

```
theta_hat_optim = opt_result.x
```

```
print("Numerical optimization MLE for the Binomial model:")  
print(f"theta_hat = {theta_hat_optim:.4f}")
```

```
Numerical optimization MLE for the Binomial model:  
theta_hat = 0.0725
```

Using the likelihood function to find the MLE of theta

```
minimize_scalar(fun, bracket=None, bounds=None, args=(), method=None, tol=None,  
options=None)
```

[\[source\]](#)

Local minimization of scalar function of one variable.

Analytic vs. Computational MLE

- Use **analytic MLE** when:
 - A closed-form solution exists (e.g., Binomial θ , Poisson rate, Normal μ and σ in the future)
- Use **computational MLE** when:
 - There is no closed-form solution
 - You're estimating multiple parameters jointly
 - You have complex models like logistic regression, mixtures, etc.

Confidence Intervals (CI)

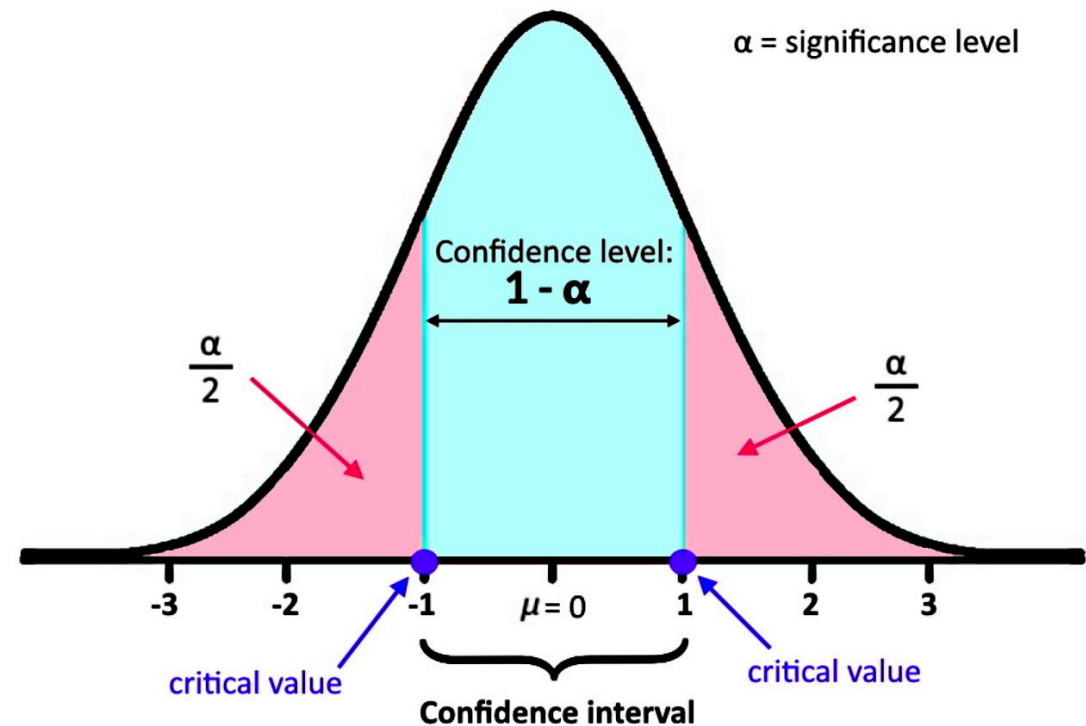
What is a Confidence Interval?

- A **confidence interval** gives us a range of values that we can treat as plausible for the true parameter.
 - A **95%** confidence interval means: If we repeated this experiment many times, about **95%** of the calculated intervals would contain the true parameter.
 - CI expresses **uncertainty about the parameter due to random sampling**.

Confidence Interval

In a two-sided confidence interval:

- α is the total probability of the tails (e.g., 0.05 for 95% CI).
- $\frac{\alpha}{2}$ is the probability in each tail (e.g., 0.025 left and right).
- The **critical value** is the point where the distribution cuts off in each tail.



CI for theta of a Binomial model

Let's say

$$y \sim \text{Binomial}(n, \theta)$$

And

$$Y = y_{obs}$$

The lower tail is

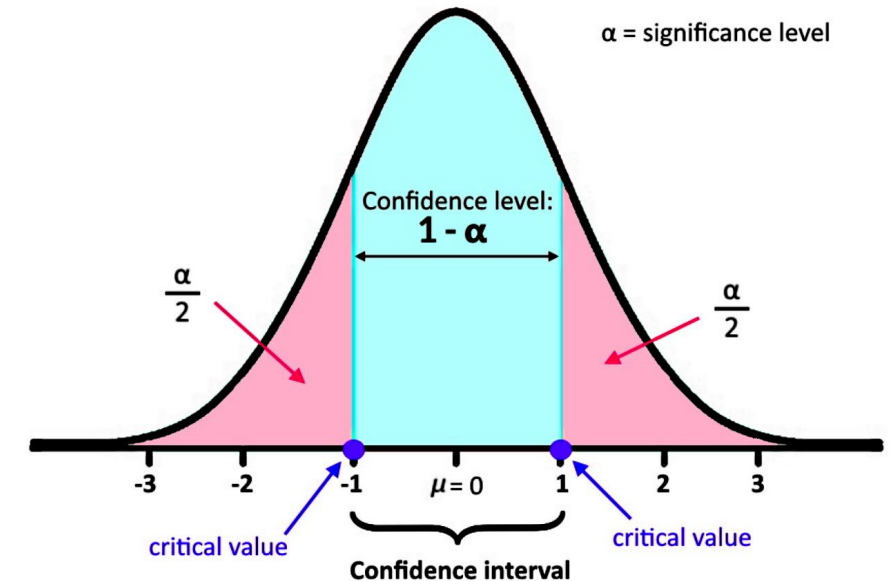
$$P(Y \leq y_{obs} \mid \theta)$$

And the upper tail is

$$P(Y \geq y_{obs} \mid \theta) = 1 - P(Y \leq y_{obs} - 1 \mid \theta)$$

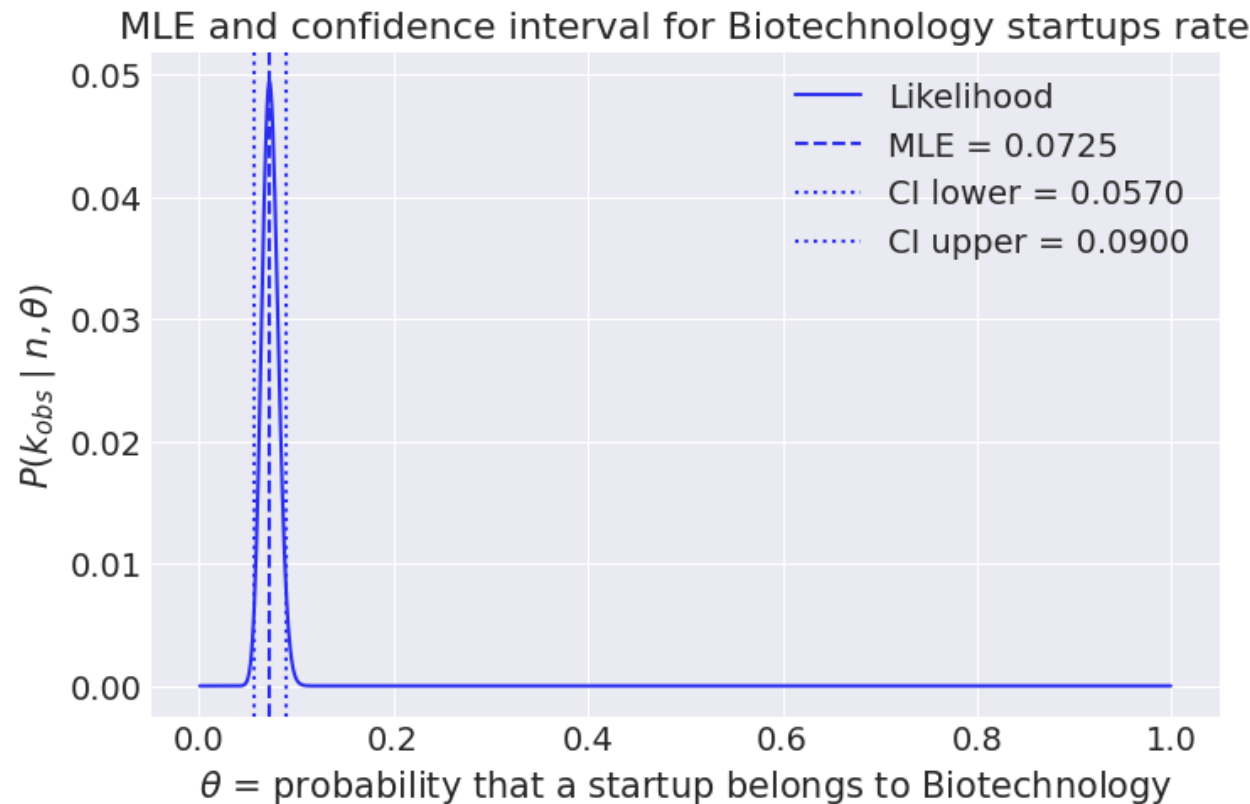
We keep the values of theta that

$$P(Y \leq y_{obs} \mid \theta) > \frac{\alpha}{2} \text{ and } P(Y \geq y_{obs} \mid \theta) > \frac{\alpha}{2}$$



CI for theta of a Binomial model

$$C_{1-\alpha} = \{\theta \in (0,1) : P(Y \leq y_{obs} \mid \theta) > \frac{\alpha}{2} \text{ and } P(Y \geq y_{obs} \mid \theta) > \frac{\alpha}{2}\}$$



Are We Right 95% of the Time?

- How often do 95% CIs for θ actually contain the true rate?
- The intervals either contain the true value or not, but across many repetitions, approximately 95% should.

